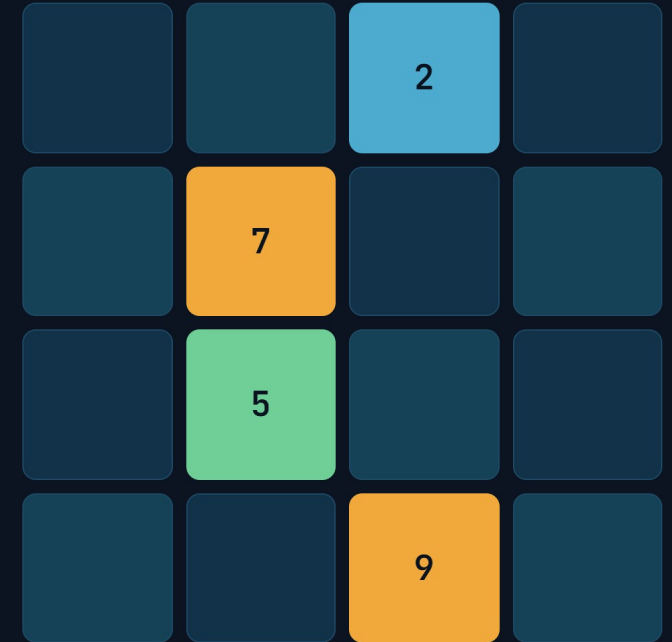


NUMPY WORKSHOP · 60 MINUTES

Mathematical Operations, Broadcasting & Reshaping

A hands-on tour of how NumPy computes fast: element-wise math, the broadcasting rules that let mismatched shapes work together, and reshaping arrays without copying data.



Python · NumPy · `import numpy as np`

[Session outline](#) → next slide

What we'll cover in 60 minutes

00 – 05	Why NumPy	Vectorization vs. Python loops, and the ndarray
05 – 20	Mathematical operations	Arithmetic, ufuncs, aggregations, comparisons
20 – 35	Broadcasting	The 3 rules, plus 3 worked examples
35 – 50	Reshaping	reshape, -1, ravel/flatten, combined with broadcasting
50 – 60	Pitfalls & cheat sheet	Views vs. copies, common errors, Q&A

Why NumPy instead of Python lists

Pure Python loop

```
result = []  
for i in range(len(a)):  
    result.append(a[i] + b[i])
```

Loops in Python, one element at a time, with interpreter overhead on every step.

NumPy, vectorized

```
result = a + b
```

The loop moves into pre-compiled C code that runs over contiguous memory.

10–100×

Typical speed-up for element-wise math on large arrays — because NumPy avoids per-element Python overhead and uses contiguous, typed memory (the **ndarray**).

The ndarray, in five attributes

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

a.shape    # (2, 3)
a.ndim     # 2
a.dtype    # dtype('int64')
a.size     # 6
```

shape

A tuple giving the size along each axis — the number that matters most for broadcasting and reshaping.

ndim

Number of axes (dimensions) — `len(shape)`.

dtype

Element type — every element shares the same type, which is why NumPy is fast.

size

Total element count, the product of shape.

Element-wise arithmetic

```
a = np.array([1, 2, 3])    b = np.array([4, 5, 6])
```

Expression	Result	What happens
<code>a + b</code>	<code>[5 7 9]</code>	Add matching positions
<code>a - b</code>	<code>[-3 -3 -3]</code>	Subtract matching positions
<code>a * b</code>	<code>[4 10 18]</code>	NOT matrix multiply
<code>a / b</code>	<code>[0.25 0.4 0.5]</code>	True division, always float
<code>a ** 2</code>	<code>[1 4 9]</code>	Scalar exponent, broadcasts
<code>a @ b</code>	<code>32</code>	Dot product – the exception

Every operator above works position-by-position on same-shaped arrays — only `@` (or `np.dot`) does linear-algebra multiplication.

Universal functions (ufuncs)

Element-wise math functions, compiled in C, applied across the whole array at once.

`np.sqrt`

```
np.sqrt([1,4,9])  
→ [1. 2. 3.]
```

`np.exp`

```
np.exp([0,1])  
→ [1. 2.718]
```

`np.log`

```
np.log([1,np.e])  
→ [0. 1.]
```

`np.abs`

```
np.abs([-3,2])  
→ [3 2]
```

`np.round`

```
np.round([1.23,4.56],1)  
→ [1.2 4.6]
```

`np.sin`

```
np.sin([0, np.pi/2])  
→ [0. 1.]
```

A ufunc takes one or more arrays and returns a new array of the same shape — no explicit loop required.

Aggregations and the axis parameter

```
m = np.array([[1,2,3],[4,5,6]])
```

1	2	3
4	5	6

shape (2, 3)

<code>m.sum()</code>	21	every element, collapses to a scalar
----------------------	----	--------------------------------------

<code>m.sum(axis=0)</code>	<code>[5 7 9]</code>	collapses rows — down each column
----------------------------	----------------------	-----------------------------------

<code>m.sum(axis=1)</code>	<code>[6 15]</code>	collapses columns — across each row
----------------------------	---------------------	-------------------------------------

Same pattern for `.mean()` `.min()` `.max()` `.std()` `.argmax()`

Comparisons produce boolean arrays

```
a = np.array([1, 2, 3, 4, 5])

a > 2
# [False False True True True]

a[a > 2]
# [3 4 5]

np.where(a > 2, a, 0)
# [0 0 3 4 5]
```

Comparison operators

> < ≥ ≤ = ≠

Combine with & | ~

(a > 1) & (a < 4)

never use and / or on arrays

Boolean indexing

a[mask] returns only the elements where the mask is True.

02

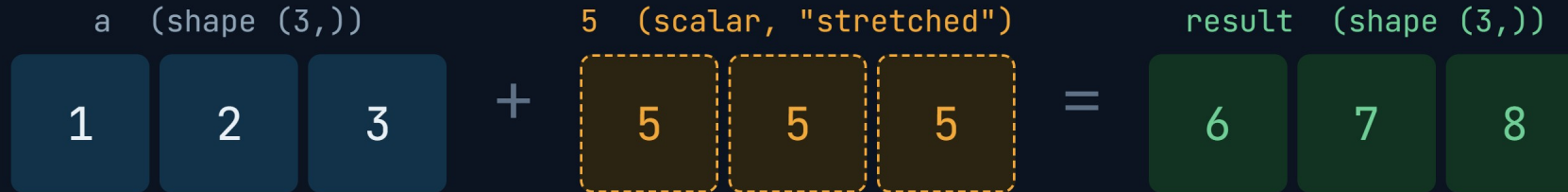
Broadcasting

How NumPy lets arrays of different shapes work together in one expression — without writing a loop, and without copying data.

BROADCASTING

What "broadcasting" means

When two arrays have different shapes, NumPy tries to "stretch" the smaller one across the larger one so their shapes match — conceptually, without ever copying the small array's data in memory.



The dashed cells are never actually created — NumPy reuses the single value 5 at almost no extra memory cost.

The three broadcasting rules

1

Align from the right

Shapes are compared starting from the trailing (rightmost) dimension, moving left.

2

Equal, or one is 1

Two dimensions are compatible when they are equal, or when one of them is 1 (it gets stretched).

3

Missing = size 1

A shorter shape is padded with 1s on the left until both shapes have the same length.

Reading the alignment for an image batch and a per-channel bias:

```
images (4d):  64  x  256  x  256  x  3
bias      (1d):                                3
result  (4d):  64  x  256  x  256  x  3
```

bias is padded to (1,1,1,3), then stretched along the first three axes.

Array + scalar: (3,) with ()

```
prices = np.array(
    [19.99, 5.50, 12.00])

taxed = prices * 1.08

print(taxed)
```

OUT

```
[21.5892  5.94    12.96   ]
```

The scalar **1.08** has shape **()**. NumPy pads it to match **(3,)** and multiplies every price by the same factor — one expression, no loop.

shapes: (3,) * () → (3,) Rule 2 in action: the scalar's missing dimension is treated as 1, then stretched to 3.

2D array + 1D array: (3,3) with (3,)



`m + row` - shapes `(3,3)` and `(3,)` align on the trailing 3, so `row` is added to every one of the 3 rows.

When shapes don't align

```
a = np.zeros((3, 4))  
b = np.array([1, 2, 3])  
  
a + b
```

```
ValueError  
operands could not be  
broadcast together with  
shapes (3,4) (3,)
```

```
a (3, 4)           trailing dim: 4  
b (3,)            trailing dim: 3  
4 ≠ 3 and neither is 1 → rule 2 fails, NumPy refuses to guess.
```

The fix: give b an explicit column axis so its shape becomes (3, 1) instead of (3,).

```
a + b.reshape(3, 1)    # works: (3,4) with (3,1) → (3,4)
```

03

Reshaping

Changing how the same data is viewed — from a flat list to a grid, or a grid to a stack of grids — without moving a single value in memory.

RESHAPING

reshape() relabels, it doesn't move data

```
a = np.arange(12)  —  shape (12,)
```

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

↓ `a.reshape(3, 4)`

0	1	2	3
4	5	6	7
8	9	10	11

```
a.reshape(3, -1)  # -1 = "figure it out": 4
```

```
a.reshape(2, 2, 3)  # any shape, same 12
```

The new shape's product must equal the array's size — 12 stays 12, however you arrange it. At most one -1 per call.

Flatten, transpose, add an axis

```
m = np.array([[1,2],[3,4],[5,6]]) # shape (3, 2)
```

Flatten to 1D

```
m.flatten()
# [1 2 3 4 5 6] — copy
```

```
m.ravel()
# [1 2 3 4 5 6] — view
```

Same result, different memory: `flatten()` always copies, `ravel()` returns a view when it can.

Transpose

```
m.T # shape (2, 3)
# [[1 3 5]
#  [2 4 6]]
```

Swaps axes; on a 2D array, rows become columns. Always a view, never a copy.

Add an axis

```
v = np.array([1,2,3])
v[:, np.newaxis]
# shape (3,1)
```

Turns a plain vector into an explicit column — exactly the fix we used for the broadcasting error.

Reshape + broadcast: center each row

3 samples, 2 features — subtract each row's own mean from that row.

```
data = np.array(
    [[1, 2], [3, 4], [5, 6]])

row_mean = data.mean(axis=1)
# [1.5, 3.5, 5.5] — shape (3,)

centered = data - row_mean.reshape(-1, 1)
```

OUT — centered

```
[[ -0.5   0.5]
 [ -0.5   0.5]
 [ -0.5   0.5]]
```

shape (3,) can't broadcast against (3,2) — trailing dims 3 and 2 don't match. reshape(-1,1) turns it into (3,1), which broadcasts across the columns.

reshape sets the shape → broadcasting decides how the math lines up: (3,2) - (3,1) → (3,2)

Four things that bite people

Views share memory

Slicing and reshape often return a **view**, not a copy. Editing the view edits the original. Call `.copy()` when you need an independent array.

reshape can force a copy

After a transpose, the data isn't laid out contiguously, so `reshape()` silently copies it. Use `.flags['C_CONTIGUOUS']` to check.

(3,) vs (3,1) broadcast differently

They look alike but align on different axes. When a result shape looks wrong, print `.shape` on both operands before the operation.

* is not matrix multiply

`*` is always element-wise. For linear-algebra multiplication use `@` or `np.matmul` — a frequent source of silent, wrong-shaped results.

Three ideas to leave with

01

Math operations

+ − ∗ / are element-wise. Use @ for matrix multiply. axis=0 walks down, axis=1 walks across.

02

Broadcasting

Align shapes from the right. Dimensions match when equal, or one of them is 1.

03

Reshaping

a.reshape(x, -1) relabels the same data. Element count never changes; use it to unlock broadcasting.

Questions?

```
import numpy as np — now go break some shapes
```